Bilkent University

Department of Electrical and Electronics Engineering

Lab 5 Report: Seven-Segment Display

Purpose

In this experiment main objective is learning operating seven-segment display on Basys 3 FPGA board. Also lab shows how to display multiple digits at same time using optical illusion called persistence of vision.

Questions

What is the internal clock frequency of Basys 3?

Built-in oscillator of Basys 3 board works at 100 MHz frequency. This means clock period is 10 nanoseconds, and it serves as primary clock input for digital circuit we designed in experiment.

How can you create a slower clock signal from this one?

To get slower clock rate, clock divider circuit can be implemented. We do this by counting rising edges of main 100 MHz clock and toggling new signal when reaching specific target count. For example in this lab, we divide 100 MHz frequency to generate 1 Hz clock, so counter increments exactly once every second.

Can you create a clock with any arbitrary frequency lower than that of the internal clock? If not, which frequencies can you create?

It is not possible to create completely arbitrary frequencies from base clock. Because new frequency depends on counting discrete clock cycles, we only produce frequencies that are exact integer divisions of main 100 MHz clock signal.

Design Specifications

Overall architecture of project has two main inputs and two outputs. Inputs are 100 MHz system clock (clk) and reset switch for initializing counter back to zero. Outputs drive display: 4-bit anode signal (an) selects active digit, and 7-bit cathode signal (cat) illuminates specific LED segments for that digit.

Design is modular, featuring top module (top_seven_segment.vhd) that connects three sub-modules:

- **Clock Divider (hex_sec_counter.vhd):** Component scales down 100 MHz board clock to practical speed. It outputs 16-bit signal (hex_out) counting up every second, and 2-bit signal (phase_count) rapidly cycling through four display digits.
- **Segment Driver (segment_driver.vhd):** Acting like multiplexer, receives phase_count and 16-bit counter value. It determines which anode to activate and routes corresponding 4 bits of counter to next stage.
- **Hexadecimal Decoder (hex_decoder.vhd):** Module takes selected 4-bit data (hex_in) and translates it to appropriate 7-bit cathode pattern (cat_out) to show hexadecimal characters on screen.

Hardware switch assigned for reset functionality. Activating switch stops system and clears screen to display zeros, deactivating it resumes upward counting process.

Methodology

Experiment started with designing counter that increments every second and displays value in hexadecimal format across four digits of Basys 3 board. To achieve this, clock divider, multiplexing driver, and hex-to-seven-segment decoder developed in VHDL. Board physically illuminates only one digit at any given instant, so design relies on switching active anodes very rapidly. Rapid switching uses persistence of vision effect, deceiving human eye to perceive all four digits as continuously lit.

After coding counter and multiplexer to handle digit selection, 4×16 decoder logic implemented for mapping 4-bit binary values to visual segment representations. Next, XDC constraint file prepared to map VHDL ports to physical pins of FPGA. Before synthesizing design, testbenches created for simulating modules and verifying logical correctness. Upon successful simulation, bitstream generated and programmed onto Basys 3 hardware to observe final counter operation.

Results

First stage of verification involved examining generated RTL schematics of overall system and sub-components. Then behavioral simulations run using testbenches. Waveform outputs confirmed main module and clock generator functioned according to theoretical expectations. (Figure 3 and Figure 4).

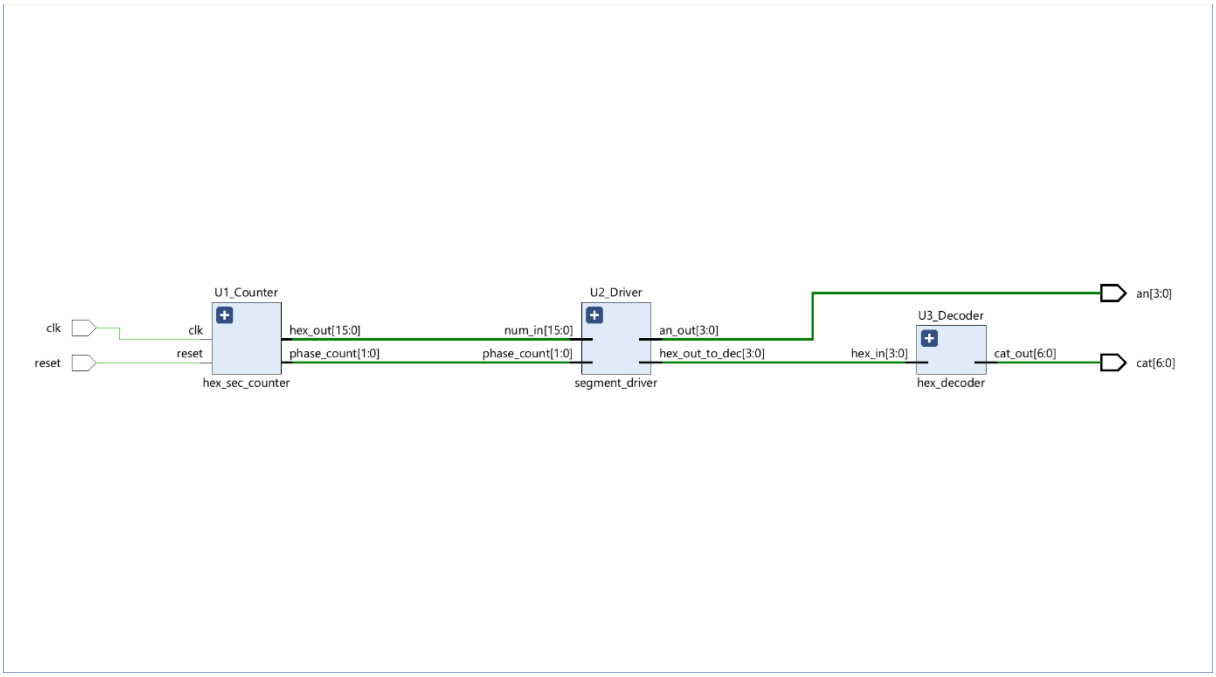


Figure 1: *RTL schematic of the top_seven_segment.vhd*

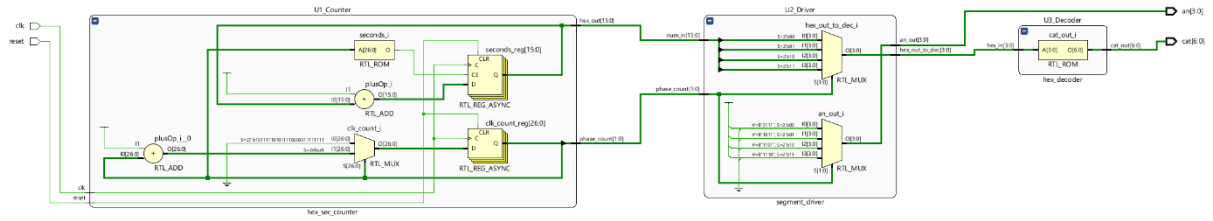


Figure 2: *RTL schematic when the sub-modules are displayed*

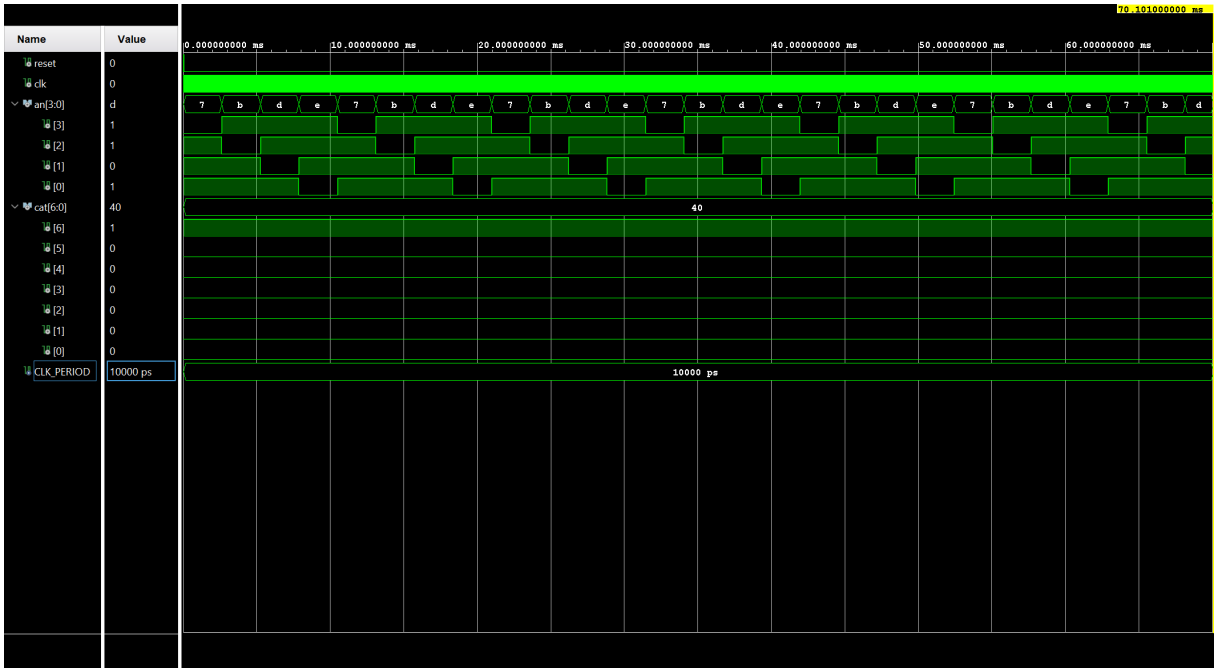


Figure 3: *Simulation Results of the Main Module*

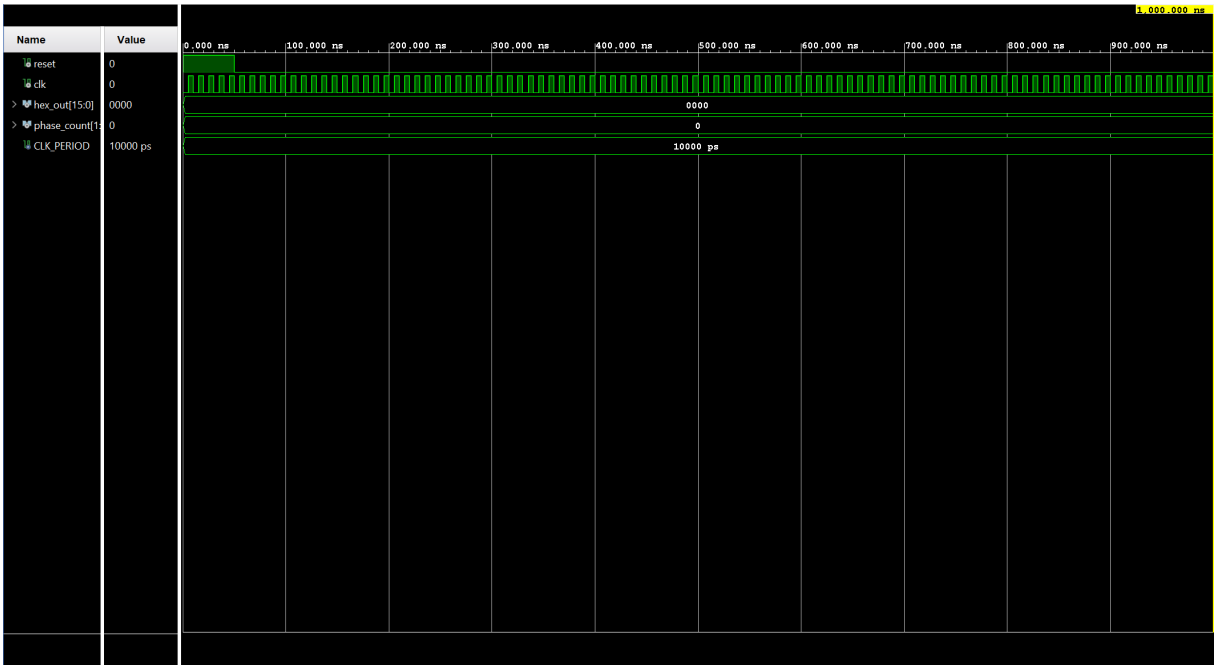


Figure 4: *Simulation Results of the Clock Signal*

Finally generated bitstream uploaded to Basys 3 board. Physical implementation mirrored simulation results successfully. When rightmost reset switch is turned on, system resets. After turning it off, timer begins counting upward sequentially from zero in

hexadecimal format. Figures 5 through 9 illustrate various moments of working hardware display over time.

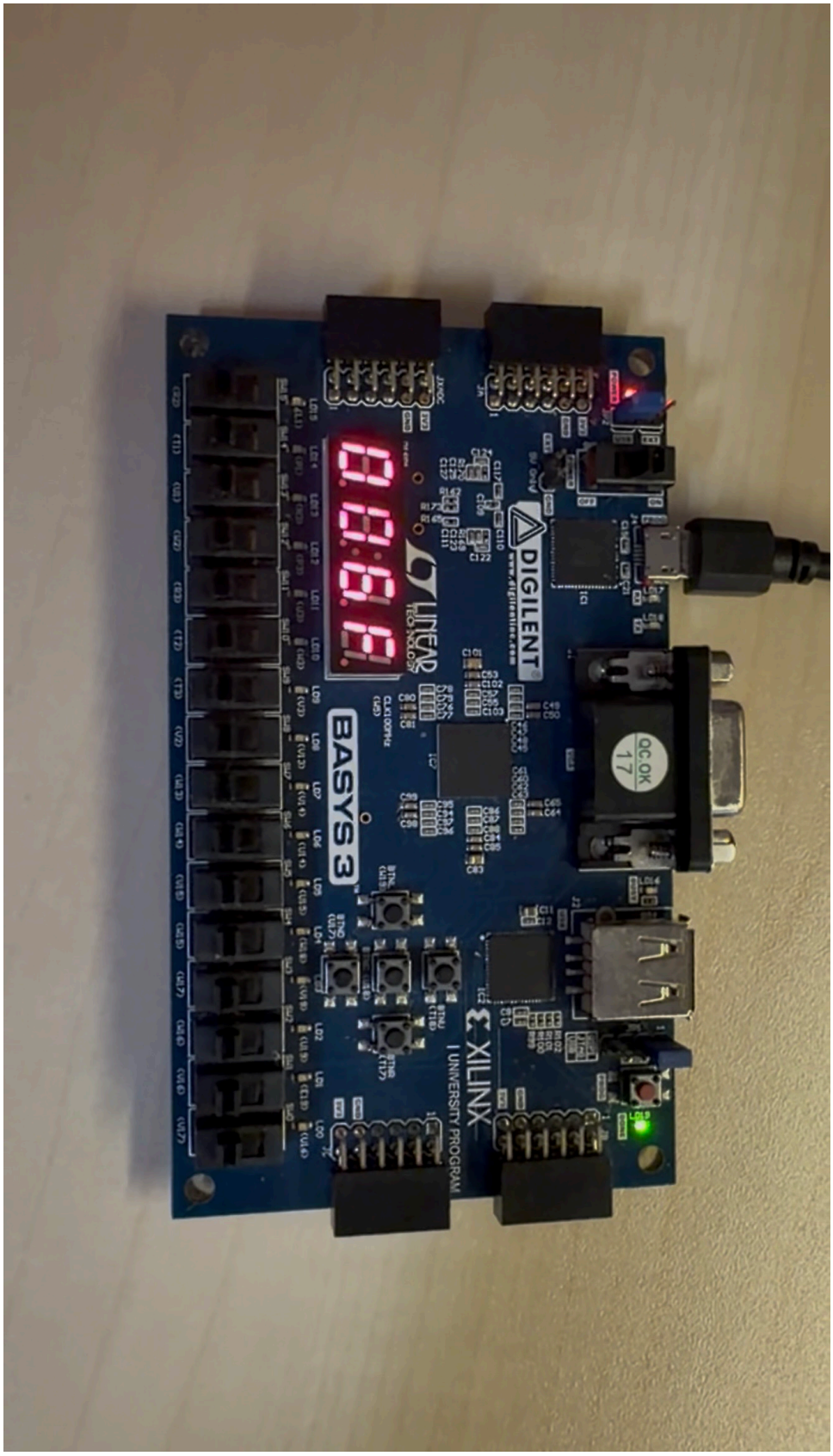


Figure 5: *Output is "006F" after 111 seconds*

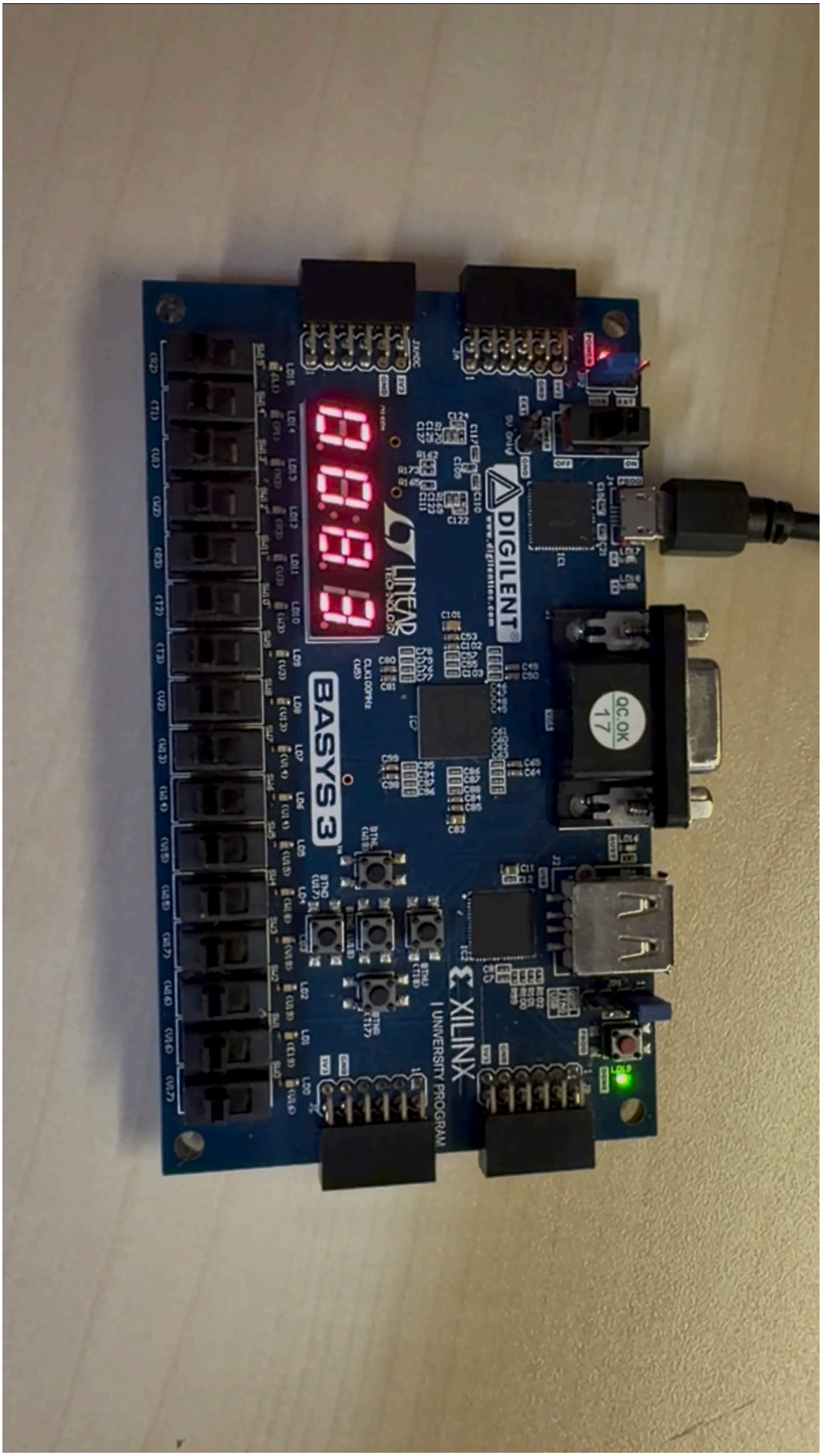


Figure 6: *Output is "0083" after 131 seconds*

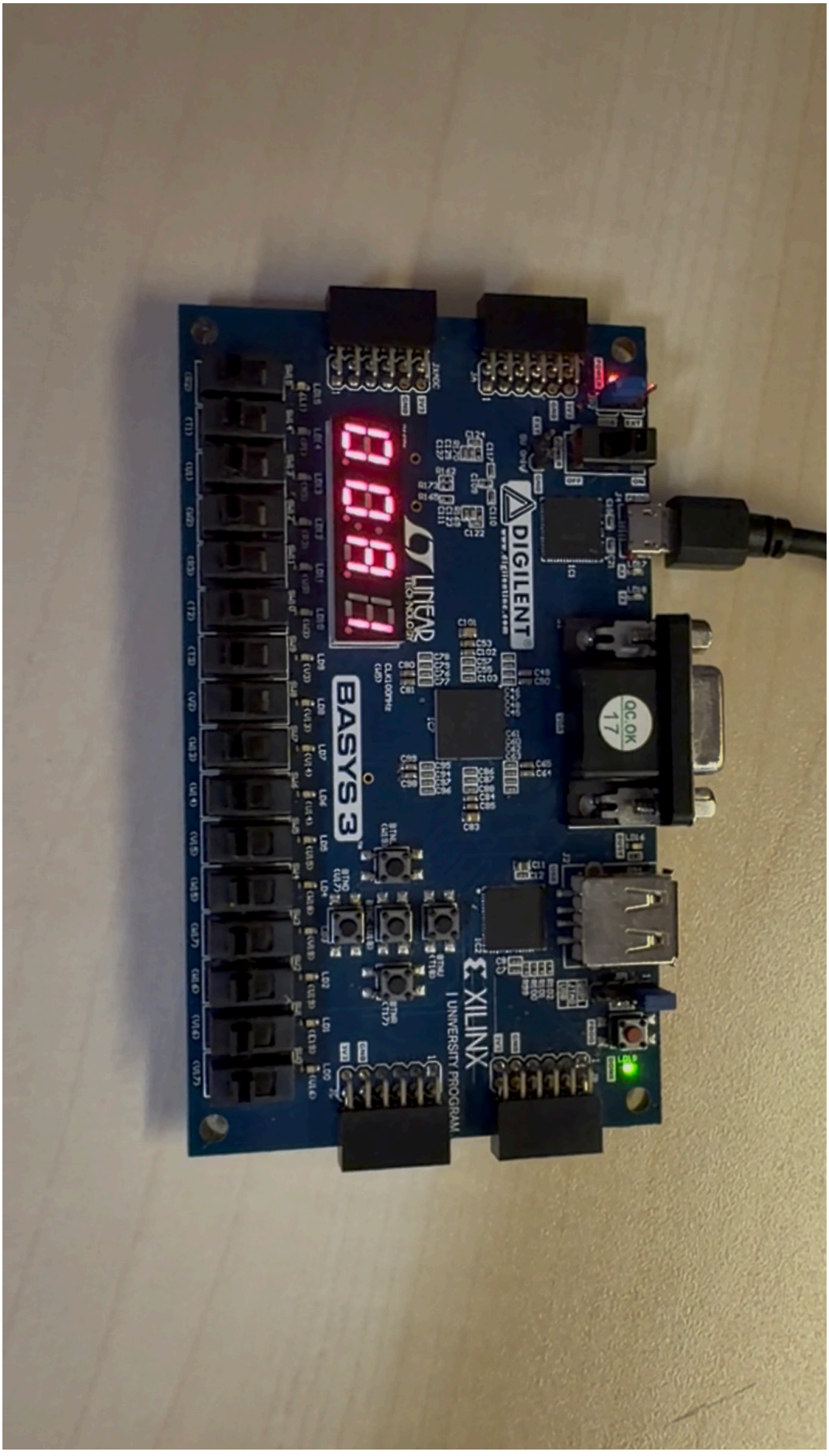


Figure 7: *Output is "00A1" after 161 seconds*

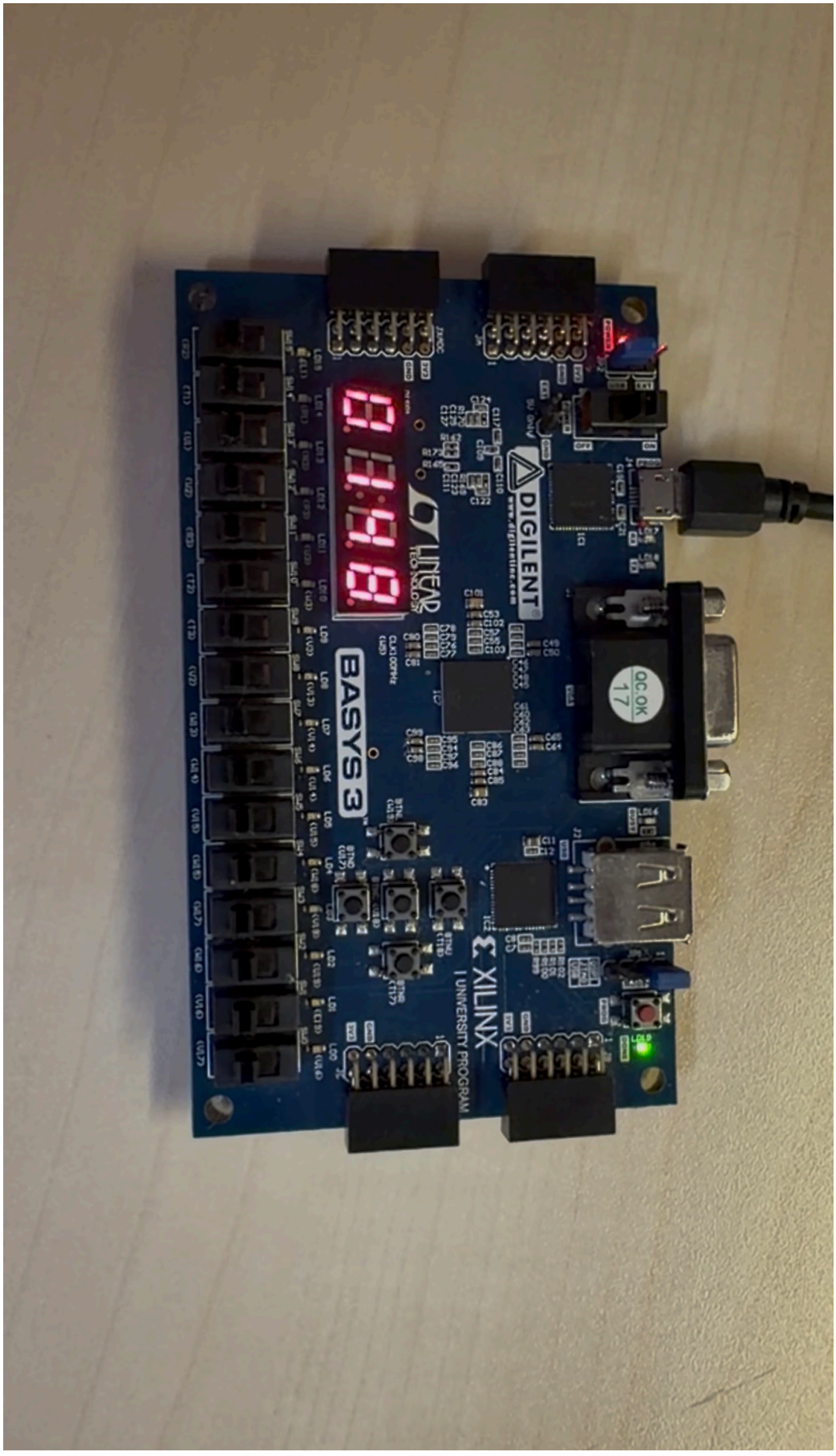


Figure 8: *Output is "0148" after 328 seconds*

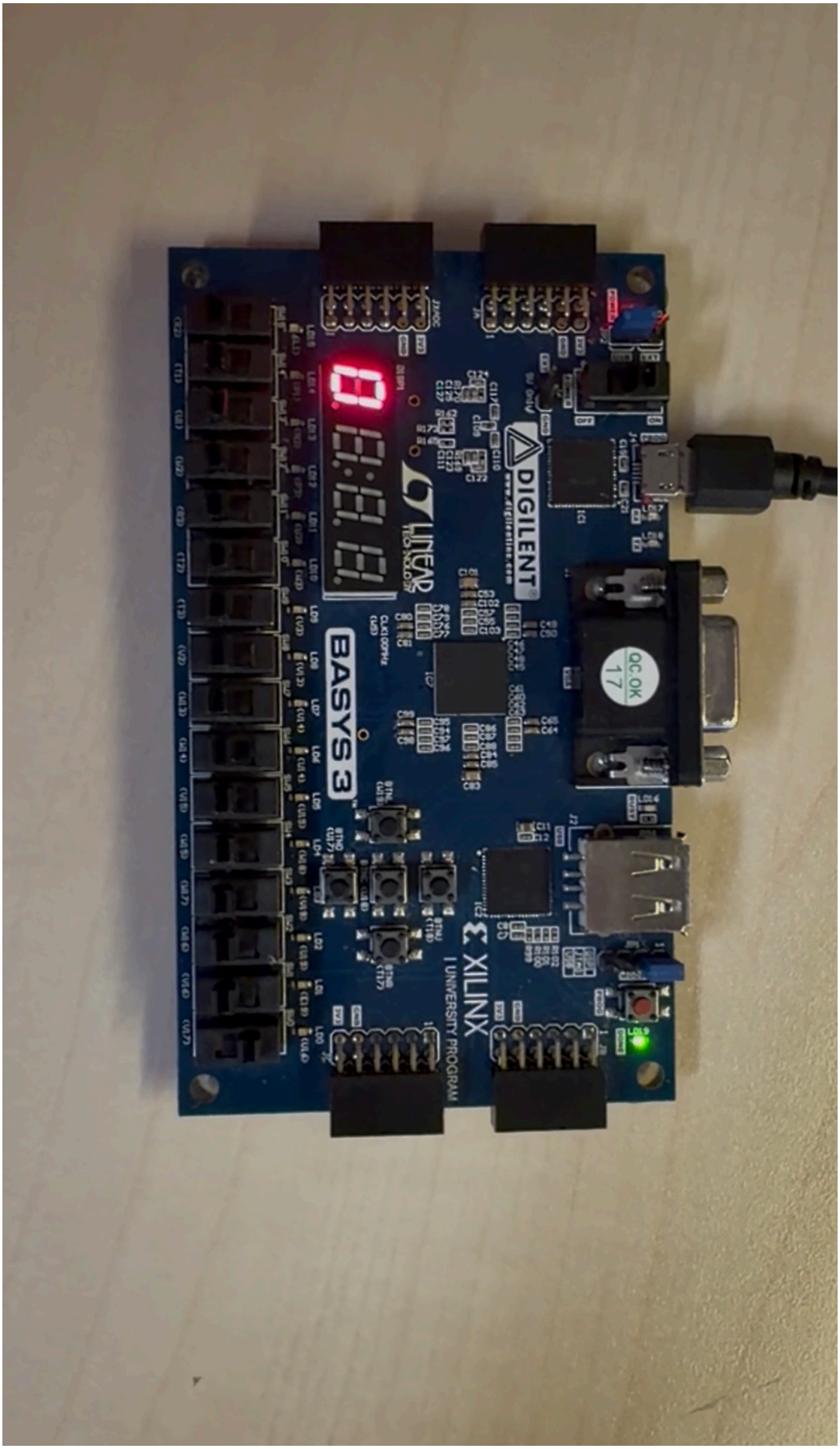


Figure 9: *Output is "0" when reset switch is on*

Conclusion

In conclusion, this laboratory experiment provided hands-on experience controlling seven-segment display through VHDL. It demonstrated practical application of clock dividers, how to manipulate 100 MHz internal oscillator of Basys 3 board to generate custom slower frequencies. Despite encountering various debugging challenges during coding phase, final hardware implementation successfully achieved desired outcome. Project effectively reinforced concept of persistence of vision, proving rapid multiplexing creates illusion of simultaneously active displays.

Appendices

top_seven_segment.vhd

```
library ieee;

use ieee.std_logic_1164.all;


entity top_seven_segment is

   port (

      clk    : in  std_logic;

      reset   : in  std_logic;

      an     : out std_logic_vector(3 downto 0);

      cat    : out std_logic_vector(6 downto 0)

   );

end entity top_seven_segment;


architecture structural of top_seven_segment is


   signal hex_val     : std_logic_vector(15 downto 0);

   signal phase_sig   : std_logic_vector(1 downto 0);

   signal hex_to_dec   : std_logic_vector(3 downto 0);
```

```
component hex_sec_counter
    port (
        clk           : in  std_logic;
        reset         : in  std_logic;
        hex_out       : out std_logic_vector(15 downto 0);
        phase_count   : out std_logic_vector(1 downto 0)
    );
end component;

component segment_driver
    port (
        phase_count   : in  std_logic_vector(1 downto 0);
        num_in        : in  std_logic_vector(15 downto 0);
        an_out        : out std_logic_vector(3 downto 0);
        hex_out_to_dec : out std_logic_vector(3 downto 0)
    );
end component;

component hex_decoder
    port (
        hex_in    : in  std_logic_vector(3 downto 0);
        cat_out   : out std_logic_vector(6 downto 0)
    );
```

```
    end component;


begin
    U1_Counter : hex_sec_counter
    port map (
        clk        => clk,
        reset      => reset,
        hex_out    => hex_val,
        phase_count => phase_sig
    );

    U2_Driver : segment_driver
    port map (
        phase_count    => phase_sig,
        num_in         => hex_val,
        an_out         => an,
        hex_out_to_dec  => hex_to_dec
    );

    U3_Decoder : hex_decoder
    port map (
        hex_in  => hex_to_dec,
        cat_out => cat
    );
```

```
end architecture structural;
```

hex_sec_counter.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity hex_sec_counter is
   port (
      clk           : in  std_logic;
      reset         : in  std_logic;
      hex_out       : out std_logic_vector(15 downto 0);
      phase_count   : out std_logic_vector(1 downto 0)
   );
end entity hex_sec_counter;

architecture behavioral of hex_sec_counter is
   constant SEC_LIMIT : natural := 99999999;
   signal clk_count   : unsigned(26 downto 0) := (others => '0');
   signal seconds     : unsigned(15 downto 0) := (others => '0');
begin
   process(clk, reset)
   begin
      if reset = '1' then
```

```
            clk_count <= (others => '0');
            seconds   <= (others => '0');
        elsif rising_edge(clk) then
            if clk_count = to_unsigned(SEC_LIMIT, clk_count'length) then
                clk_count <= (others => '0');
                seconds <= seconds + 1;
            else
                clk_count <= clk_count + 1;
            end if;
        end if;
    end process;

    hex_out <= std_logic_vector(seconds);
    phase_count <= std_logic_vector(clk_count(19 downto 18));

end architecture behavioral;
```

segment_driver.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity segment_driver is
    port (
        phase_count     : in  std_logic_vector(1 downto 0);
```

```
        num_in         : in  std_logic_vector(15 downto 0);
        an_out         : out std_logic_vector(3 downto 0);
        hex_out_to_dec  : out std_logic_vector(3 downto 0)
    );
end entity segment_driver;

architecture behavioral of segment_driver is
begin
    process(phase_count, num_in)
    begin
        case phase_count is
            when "00" =>
                an_out <= "0111";
                hex_out_to_dec <= num_in(15 downto 12);
            when "01" =>
                an_out <= "1011";
                hex_out_to_dec <= num_in(11 downto 8);
            when "10" =>
                an_out <= "1101";
                hex_out_to_dec <= num_in(7 downto 4);
            when "11" =>
                an_out <= "1110";
                hex_out_to_dec <= num_in(3 downto 0);
            when others =>
```

```
            an_out <= "1111";
            hex_out_to_dec <= (others => '0');
        end case;
    end process;
end architecture behavioral;
```

hex_decoder.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity hex_decoder is
    port (
        hex_in      : in  std_logic_vector(3 downto 0);
        cat_out     : out std_logic_vector(6 downto 0)
    );
end entity hex_decoder;

architecture behavioral of hex_decoder is
begin
    process(hex_in)
    begin
        case hex_in is
            when "0000" => cat_out <= "1000000";
            when "0001" => cat_out <= "1111001";
```

```
            when "0010" => cat_out <= "0100100";
            when "0011" => cat_out <= "0110000";
            when "0100" => cat_out <= "0011001";
            when "0101" => cat_out <= "0010010";
            when "0110" => cat_out <= "0000010";
            when "0111" => cat_out <= "1111000";
            when "1000" => cat_out <= "0000000";
            when "1001" => cat_out <= "0010000";
            when "1010" => cat_out <= "0001000";
            when "1011" => cat_out <= "0000011";
            when "1100" => cat_out <= "1000110";
            when "1101" => cat_out <= "0100001";
            when "1110" => cat_out <= "0000110";
            when "1111" => cat_out <= "0001110";
            when others => cat_out <= "1111111";
        end case;
    end process;
end architecture behavioral;
```

tb_top_seven_segment.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

```
entity tb_top_seven_segment is
end tb_top_seven_segment;

architecture Behavioral of tb_top_seven_segment is
    constant CLK_PERIOD: time := 10 ns;
    signal reset: std_logic;
    signal clk: std_logic;
    signal an : std_logic_vector(3 downto 0);
    signal cat: std_logic_vector(6 downto 0);
begin
    dut: entity work.top_seven_segment(structural)
    port map (
        clk => clk,
        reset => reset,
        an => an,
        cat => cat
    );

    clock_process: process
    begin
        clk <= '0';
        wait for CLK_PERIOD/2;
        clk <= '1';
        wait for CLK_PERIOD/2;
```

```
    end process;

    sim_process: process
    begin
        reset <= '1';
        wait for 30 ns;
        reset <= '0';
        wait for 100 ns;
        wait for 100 ms;
        wait;
    end process;
end Behavioral;
```

tb_counter.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_counter is
end tb_counter;

architecture Behavioral of tb_counter is
    constant CLK_PERIOD: time := 10 ns;
    signal reset : std_logic;
```

```
    signal clk : std_logic;
    signal hex_out : std_logic_vector(15 downto 0);
    signal phase_count : std_logic_vector(1 downto 0);
begin
    dut: entity work.hex_sec_counter(behavioral)
    port map (
        clk => clk,
        reset => reset,
        hex_out => hex_out,
        phase_count => phase_count
    );

    clock_process: process
    begin
        clk <= '0';
        wait for CLK_PERIOD/2;
        clk <= '1';
        wait for CLK_PERIOD/2;
    end process;

    sim_process: process
    begin
        reset <= '1';
        wait for 50 ns;
```

```
        reset <= '0';
        wait for 50 ns;
        wait for 150 ms;
        wait;
    end process;
end Behavioral;
```

constraint.xdc

```
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

set_property PACKAGE_PIN V17 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]

set_property PACKAGE_PIN U2 [get_ports {an[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
```

```
set_property PACKAGE_PIN W7 [get_ports {cat[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {cat[0]}]

set_property PACKAGE_PIN W6 [get_ports {cat[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {cat[1]}]

set_property PACKAGE_PIN U8 [get_ports {cat[2]}]

set_property IOSTANDARD LVCMOS33 [get_ports {cat[2]}]

set_property PACKAGE_PIN V8 [get_ports {cat[3]}]

set_property IOSTANDARD LVCMOS33 [get_ports {cat[3]}]

set_property PACKAGE_PIN U5 [get_ports {cat[4]}]

set_property IOSTANDARD LVCMOS33 [get_ports {cat[4]}]

set_property PACKAGE_PIN V5 [get_ports {cat[5]}]

set_property IOSTANDARD LVCMOS33 [get_ports {cat[5]}]

set_property PACKAGE_PIN U7 [get_ports {cat[6]}]

set_property IOSTANDARD LVCMOS33 [get_ports {cat[6]}]
```

References

[1] Bilkent University, Department of Electrical and Electronics Engineering. Lab 5: Seven-Segment Display.

[2] Digilent Inc. Basys 3 FPGA Board Reference Manual. Available: https://digilent.com/reference/programmable-logic/basys-3/reference-manual

[3] "Persistence of vision," Wikipedia, The Free Encyclopedia. Available: https://en.wikipedia.org/wiki/Persistence_of_vision

[4] "Seven-segment display," Wikipedia, The Free Encyclopedia. Available: https://en.wikipedia.org/wiki/Seven-segment_display

[5] Digilent Inc. Basys 3 Master XDC Constraint File (basys3xdc.pdf).

[6] Instructables. "Seven Segment Display Tutorial". Available: https://www.instructables.com/Seven-Segment-Display-Tutorial/

[7] D. Madison. "LED-Segment-ASCII," GitHub Repository. Available: https://github.com/dmadison/LED-Segment-ASCII

[8] M. Morris Mano and M. D. Ciletti. Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed. Pearson Education.

[9] Xilinx Inc. Vivado Design Suite User Guide: Logic Simulation.

[10] IEEE Computer Society. IEEE Standard VHDL Language Reference Manual, IEEE Std 1076-2008.